

class-connect- - PDF 1 - backend

Source-code export | 2026-09-10 | landscape A4 | monospace | selectable text

Contents:

```
=== backend/src/index.js ===  
=== backend/wrangler.toml ===  
=== backend/migrations/0001_initial.sql ===
```

Line numbers shown at left; long lines wrap with a '+' continuation marker. No secrets are included.

=== backend/src/index.js === (1086 lines, 31819 bytes)

```
1 | const SESSION_SECONDS = 7 * 24 * 60 * 60;
2 | const MAX_FILE_BYTES = 5 * 1024 * 1024;
3 |
4 | const ALLOWED_MIME = new Set([
5 |   "image/jpeg",
6 |   "image/png",
7 |   "image/webp",
8 |   "application/pdf",
9 |   "audio/webm",
10 |   "audio/ogg"
11 | ]);
12 |
13 | const encoder = new TextEncoder();
14 |
15 | class HttpError extends Error {
16 |   constructor(status, message) {
17 |     super(message);
18 |     this.status = status;
19 |   }
20 | }
21 |
22 | function fail(status, message) {
23 |   throw new HttpError(status, message);
24 | }
25 |
26 | function json(value, status = 200, extra = {}) {
27 |   return new Response(JSON.stringify(value), {
28 |     status,
29 |     headers: {
30 |       "Content-Type": "application/json; charset=utf-8",
31 |       "Cache-Control": "no-store",
32 |       "X-Content-Type-Options": "nosniff",
33 |       ...extra
34 |     }
35 |   });
36 | }
37 |
38 | function ErrorResponse(error) {
39 |   if (!(error instanceof HttpError)) {
40 |     console.error("Request failed", error?.stack || error);
41 |   }
42 |
43 |   return json(
44 |     {
45 |       error:
46 |         error instanceof HttpError
47 |           ? error.message
48 |           : "Server error. Please retry."
49 |     },
50 |     error instanceof HttpError ? error.status : 500
51 |   );
52 | }
53 |
54 | export function cleanText(value, min, max, label) {
55 |   if (typeof value !== "string") fail(400, `${label} is required.`);
56 |
57 |   const result = value.trim();
```

```

58 |
59 |   if (result.length < min || result.length > max) {
60 |     fail(400, `${label} must contain ${min}-${max} characters.`);
61 |   }
62 |
63 |   return result;
64 | }
65 |
66 | export function normalizeInvite(value) {
67 |   return cleanText(value, 8, 100, "Invite code")
68 |     .replace(/[\s-]/g, "")
69 |     .toUpperCase();
70 | }
71 |
72 | export function allowedMime(value) {
73 |   const mime = String(value || "").split(";")[0].trim().toLowerCase();
74 |
75 |   if (!ALLOWED_MIME.has(mime)) {
76 |     fail(400, "Supported files: JPEG, PNG, WebP, PDF, WebM audio or Ogg audio.");
77 |   }
78 |
79 |   return mime;
80 | }
81 |
82 | function randomToken(bytes = 32) {
83 |   const data = crypto.getRandomValues(new Uint8Array(bytes));
84 |   return Array.from(data, byte => byte.toString(16).padStart(2, "0")).join("");
85 | }
86 |
87 | async function hash(value) {
88 |   const digest = await crypto.subtle.digest("SHA-256", encoder.encode(value));
89 |   return Array.from(new Uint8Array(digest), byte =>
90 |     byte.toString(16).padStart(2, "0")
91 |   ).join("");
92 | }
93 |
94 | async function boundedBytes(request, limit) {
95 |   if (!request.body) return new Uint8Array();
96 |
97 |   const reader = request.body.getReader();
98 |   const chunks = [];
99 |   let length = 0;
100 |
101 |   try {
102 |     while (true) {
103 |       const { done, value } = await reader.read();
104 |       if (done) break;
105 |
106 |       length += value.byteLength;
107 |
108 |       if (length > limit) {
109 |         await reader.cancel();
110 |         fail(413, "Request is too large.");
111 |       }
112 |
113 |       chunks.push(value);
114 |     }
115 |   } finally {
116 |     reader.releaseLock();

```

```

117 |   }
118 |
119 |   const output = new Uint8Array(length);
120 |   let offset = 0;
121 |
122 |   for (const chunk of chunks) {
123 |     output.set(chunk, offset);
124 |     offset += chunk.byteLength;
125 |   }
126 |
127 |   return output;
128 | }
129 |
130 | async function bodyJSON(request, limit = 8192) {
131 |   try {
132 |     const bytes = await boundedBytes(request, limit);
133 |     const value = JSON.parse(new TextDecoder().decode(bytes));
134 |
135 |     if (!value || typeof value !== "object" || Array.isArray(value)) {
136 |       fail(400, "Expected a JSON object.");
137 |     }
138 |
139 |     return value;
140 |   } catch (error) {
141 |     if (error instanceof HttpError) throw error;
142 |     fail(400, "Invalid JSON.");
143 |   }
144 | }
145 |
146 | function cookieName(env) {
147 |   return env.APP_ORIGIN.startsWith("https:")
148 |     ? "__Host-classconnect"
149 |     : "classconnect-dev";
150 | }
151 |
152 | function sessionCookie(env, token, maxAge = SESSION_SECONDS) {
153 |   const secure = env.APP_ORIGIN.startsWith("https:") ? "; Secure" : "";
154 |
155 |   return `${cookieName(env)}=${token}; Path=/; HttpOnly; SameSite=Strict; Max-Age=${maxAge}${secure}`;
156 | }
157 |
158 | function getCookie(request, name) {
159 |   for (const part of (request.headers.get("Cookie") || "").split(";")) {
160 |     const index = part.indexOf("=");
161 |     if (index < 0) continue;
162 |
163 |     if (part.slice(0, index).trim() === name) {
164 |       return part.slice(index + 1).trim();
165 |     }
166 |   }
167 |
168 |   return "";
169 | }
170 |
171 | async function authenticate(request, env) {
172 |   const token = getCookie(request, cookieName(env));
173 |
174 |   if (!/^[a-f0-9]{64}$/.test(token)) fail(401, "Please sign in.");
175 |

```

```

176 |   const tokenHash = await hash(token);
177 |
178 |   const user = await env.DB.prepare(`
179 |     SELECT u.id, u.display_name, u.role, s.expires_at
180 |     FROM sessions s
181 |     JOIN users u ON u.id = s.user_id
182 |     WHERE s.token_hash = ? AND s.expires_at > ?
183 | `).bind(tokenHash, Date.now()).first();
184 |
185 |   if (!user) fail(401, "Your session has expired. Please sign in.");
186 |
187 |   return { ...user, session_hash: tokenHash };
188 | }
189 |
190 | async function createSession(env, userId) {
191 |   const token = randomToken();
192 |   const expires = Date.now() + SESSION_SECONDS * 1000;
193 |
194 |   await env.DB.prepare(`
195 |     INSERT INTO sessions(token_hash, user_id, expires_at)
196 |     VALUES (?, ?, ?)
197 | `).bind(await hash(token), userId, expires).run();
198 |
199 |   return sessionCookie(env, token);
200 | }
201 |
202 | function publicUser(user) {
203 |   return {
204 |     id: user.id,
205 |     display_name: user.display_name,
206 |     role: user.role
207 |   };
208 | }
209 |
210 | function verifyOrigin(request, env) {
211 |   const url = new URL(request.url);
212 |
213 |   if (url.origin !== env.APP_ORIGIN) {
214 |     fail(403, "Unrecognized application origin.");
215 |   }
216 |
217 |   const unsafe = !["GET", "HEAD", "OPTIONS"].includes(request.method);
218 |   const websocket = request.headers.get("Upgrade")?.toLowerCase() === "websocket";
219 |
220 |   if (
221 |     (unsafe || websocket) &&
222 |     request.headers.get("Origin") !== env.APP_ORIGIN
223 |   ) {
224 |     fail(403, "Origin verification failed.");
225 |   }
226 | }
227 |
228 | async function membership(env, classId, userId) {
229 |   return env.DB.prepare(`
230 |     SELECT m.*, c.teacher_id, c.name
231 |     FROM class_members m
232 |     JOIN classes c ON c.id = m.class_id
233 |     WHERE m.class_id = ? AND m.user_id = ?
234 | `).bind(classId, userId).first();

```

```

235 | }
236 |
237 | async function consume(env, key, limit, seconds, amount = 1) {
238 |   const stub = env.GUARD.get(env.GUARD.idFromName("global"));
239 |
240 |   const response = await stub.fetch("https://internal/consume", {
241 |     method: "POST",
242 |     body: JSON.stringify({ key, limit, seconds, amount })
243 |   });
244 |
245 |   if (!response.ok) {
246 |     fail(429, "Application quota reached. Please try again later.");
247 |   }
248 | }
249 |
250 | async function iplimit(request, env, category, limit, seconds) {
251 |   const ip = request.headers.get("CF-Connecting-IP") || "local";
252 |   const identity = (await hash(ip)).slice(0, 24);
253 |
254 |   await consume(env, `ip:${category}:${identity}`, limit, seconds);
255 | }
256 |
257 | async function roomRequest(request, env, classId) {
258 |   const stub = env.ROOMS.get(env.ROOMS.idFromName(classId));
259 |   return stub.fetch(request);
260 | }
261 |
262 | function attachmentInfo(row) {
263 |   if (!row.attachment_id || !row.file_name) return null;
264 |
265 |   return {
266 |     id: row.attachment_id,
267 |     file_name: row.file_name,
268 |     mime_type: row.mime_type,
269 |     size_bytes: row.size_bytes
270 |   };
271 | }
272 |
273 | function messageJSON(row) {
274 |   return {
275 |     seq: row.seq,
276 |     id: row.id,
277 |     class_id: row.class_id,
278 |     sender_id: row.sender_id,
279 |     sender_name: row.sender_name,
280 |     client_id: row.client_id,
281 |     content: row.deleted ? "" : row.content,
282 |     created_at: row.created_at,
283 |     deleted: Boolean(row.deleted),
284 |     attachment: row.deleted ? null : attachmentInfo(row)
285 |   };
286 | }
287 |
288 | const MESSAGE_SELECT = `
289 |   SELECT m.*, u.display_name AS sender_name,
290 |     a.file_name, a.mime_type, a.size_bytes
291 |   FROM messages m
292 |   JOIN users u ON u.id = m.sender_id
293 |   LEFT JOIN attachments a ON a.id = m.attachment_id

```

```

294 | `;
295 |
296 | async function loadMessage(env, id) {
297 |   const row = await env.DB.prepare(
298 |     `${MESSAGE_SELECT} WHERE m.id = ?`
299 |   ).bind(id).first();
300 |
301 |   return row ? messageJSON(row) : null;
302 | }
303 |
304 | function sniffFile(bytes, mime) {
305 |   const ascii = (start, length) =>
306 |     String.fromCharCode(...bytes.slice(start, start + length));
307 |
308 |   const valid =
309 |     (mime === "image/jpeg" &&
310 |      bytes[0] === 0xff && bytes[1] === 0xd8 && bytes[2] === 0xff) ||
311 |     (mime === "image/png" &&
312 |      [137, 80, 78, 71, 13, 10, 26, 10].every((b, i) => bytes[i] === b)) ||
313 |     (mime === "image/webp" &&
314 |      ascii(0, 4) === "RIFF" && ascii(8, 4) === "WEBP") ||
315 |     (mime === "application/pdf" && ascii(0, 5) === "%PDF-") ||
316 |     (mime === "audio/ogg" && ascii(0, 4) === "OggS") ||
317 |     (mime === "audio/webm" &&
318 |      [0x1a, 0x45, 0xdf, 0xa3].every((b, i) => bytes[i] === b));
319 |
320 |   if (!valid) fail(400, "File contents do not match its declared format.");
321 | }
322 |
323 | async function upload(request, env, user) {
324 |   const url = new URL(request.url);
325 |   const classId = cleanText(url.searchParams.get("class"), 1, 100, "Class");
326 |   const fileName = cleanText(url.searchParams.get("name"), 1, 160, "File name");
327 |   const size = Number(url.searchParams.get("size"));
328 |   const mime = allowedMime(request.headers.get("Content-Type"));
329 |
330 |   if (!Number.isInteger(size) || size < 1 || size > MAX_FILE_BYTES) {
331 |     fail(413, "Files must be between 1 byte and 5 MiB.");
332 |   }
333 |
334 |   const member = await membership(env, classId, user.id);
335 |   if (!member || member.muted) fail(403, "You cannot upload to this class.");
336 |
337 |   // Reservations are not refunded on failure. This is intentionally conservative.
338 |   await consume(env, `upload-count:${user.id}`, 10, 86400);
339 |   await consume(env, `upload-bytes:${user.id}`, 20 * 1024 * 1024, 86400, size);
340 |   await consume(env, "upload-global-day", 50 * 1024 * 1024, 86400, size);
341 |   await consume(env, "upload-lifetime", 512 * 1024 * 1024, 0, size);
342 |
343 |   const bytes = await boundedBytes(request, MAX_FILE_BYTES);
344 |   if (bytes.byteLength !== size) fail(400, "Upload size mismatch.");
345 |
346 |   sniffFile(bytes, mime);
347 |
348 |   // Recheck after reading the body in case moderation changed while uploading.
349 |   const current = await membership(env, classId, user.id);
350 |   if (!current || current.muted) fail(403, "Upload permission was revoked.");
351 |
352 |   const id = crypto.randomUUID();

```

```

353 |   const key = `${classId}/${id}`;
354 |
355 |   await env.FILES.put(key, bytes, {
356 |     httpMetadata: { contentType: mime }
357 |   });
358 |
359 |   try {
360 |     await env.DB.prepare(`
361 |       INSERT INTO attachments
362 |         (id, class_id, owner_id, r2_key, file_name, mime_type, size_bytes, created_at)
363 |       VALUES (?, ?, ?, ?, ?, ?, ?, ?)
364 |     `).bind(
365 |       id, classId, user.id, key, fileName, mime, size, Date.now()
366 |     ).run();
367 |   } catch (error) {
368 |     await env.FILES.delete(key);
369 |     throw error;
370 |   }
371 |
372 |   return json({ id, file_name: fileName, mime_type: mime, size_bytes: size }, 201);
373 | }
374 |
375 | async function download(request, env, user, attachmentId) {
376 |   await consume(env, `downloads:${user.id}`, 120, 3600);
377 |
378 |   const row = await env.DB.prepare(`
379 |     SELECT a.*
380 |     FROM attachments a
381 |     JOIN messages m ON m.attachment_id = a.id AND m.deleted = 0
382 |     JOIN class_members cm ON cm.class_id = a.class_id AND cm.user_id = ?
383 |     WHERE a.id = ?
384 |   `).bind(user.id, attachmentId).first();
385 |
386 |   if (!row) fail(404, "Attachment is unavailable.");
387 |
388 |   const object = await env.FILES.get(row.r2_key);
389 |   if (!object) fail(404, "Attachment has expired.");
390 |
391 |   const fileName = encodeURIComponent(row.file_name).replace(
392 |     /['()]/g,
393 |     char => `%${char.charCodeAt(0).toString(16).toUpperCase()}`
394 |   );
395 |
396 |   return new Response(object.body, {
397 |     headers: {
398 |       "Content-Type": row.mime_type,
399 |       "Content-Length": String(object.size),
400 |       "Content-Disposition": `attachment; filename*=UTF-8'${fileName}`,
401 |       "Cache-Control": "private, no-store",
402 |       "X-Content-Type-Options": "nosniff",
403 |       "Content-Security-Policy": "sandbox"
404 |     }
405 |   });
406 | }
407 |
408 | async function handle(request, env) {
409 |   verifyOrigin(request, env);
410 |
411 |   const url = new URL(request.url);

```

```

412 | const path = url.pathname;
413 | const method = request.method;
414 |
415 | if (path === "/api/health" && method === "GET") {
416 |   return json({
417 |     ok: true,
418 |     features: { push: false, calls: false, directMessages: false }
419 |   });
420 | }
421 |
422 | if (path === "/api/auth/login" && method === "POST") {
423 |   await ipLimit(request, env, "login", 20, 3600);
424 |
425 |   const body = await bodyJSON(request);
426 |   const key = cleanText(body.key, 64, 64, "Recovery key").toLowerCase();
427 |
428 |   if (!/^[a-f0-9]{64}$/.test(key)) fail(401, "Invalid recovery key.");
429 |
430 |   const user = await env.DB.prepare(`
431 |     SELECT id, display_name, role FROM users WHERE recovery_hash = ?
432 |   `).bind(await hash(key)).first();
433 |
434 |   if (!user) fail(401, "Invalid recovery key.");
435 |
436 |   return json(
437 |     { user: publicUser(user) },
438 |     200,
439 |     { "Set-Cookie": await createSession(env, user.id) }
440 |   );
441 | }
442 |
443 | if (path === "/api/auth/join" && method === "POST") {
444 |   await ipLimit(request, env, "join", 10, 3600);
445 |   await consume(env, "enrollments-global", 100, 86400);
446 |
447 |   const body = await bodyJSON(request);
448 |   const displayName = cleanText(body.name, 1, 60, "Name");
449 |   const inviteHash = await hash(normalizeInvite(body.code));
450 |
451 |   const classroom = await env.DB.prepare(`
452 |     SELECT id FROM classes WHERE join_hash = ?
453 |   `).bind(inviteHash).first();
454 |
455 |   if (!classroom) fail(400, "Invalid invitation code.");
456 |
457 |   const userId = crypto.randomUUID();
458 |   const recoveryKey = randomToken();
459 |
460 |   // Guarded INSERT prevents enrollment if the code was rotated meanwhile
461 |   // or the class has reached its member cap.
462 |   const results = await env.DB.batch([
463 |     env.DB.prepare(`
464 |       INSERT INTO users(id, display_name, role, recovery_hash, created_at)
465 |       SELECT ?, ?, 'student', ?, ?
466 |       WHERE EXISTS (
467 |         SELECT 1 FROM classes c
468 |         WHERE c.id = ? AND c.join_hash = ?
469 |         AND (SELECT COUNT(*) FROM class_members WHERE class_id = c.id) < 41
470 |       )

```

```

471 |     `).bind(
472 |       userId, displayName, await hash(recoveryKey), Date.now(),
473 |       classroom.id, inviteHash
474 |     ),
475 |     env.DB.prepare(`
476 |       INSERT INTO class_members(class_id, user_id, joined_at)
477 |       SELECT ?, id, ? FROM users WHERE id = ?
478 |     `).bind(classroom.id, Date.now(), userId)
479 |   ]);
480 |
481 |   if (!results[0].meta.changes) {
482 |     fail(409, "Invitation changed or the class is full.");
483 |   }
484 |
485 |   return json(
486 |     {
487 |       user: { id: userId, display_name: displayName, role: "student" },
488 |       recoveryKey
489 |     },
490 |     201,
491 |     { "Set-Cookie": await createSession(env, userId) }
492 |   );
493 | }
494 |
495 | const user = await authenticate(request, env);
496 |
497 | if (path === "/api/auth/logout" && method === "POST") {
498 |   await env.DB.prepare(
499 |     "DELETE FROM sessions WHERE token_hash = ?"
500 |   ).bind(user.session_hash).run();
501 |
502 |   return json(
503 |     { ok: true },
504 |     200,
505 |     { "Set-Cookie": sessionCookie(env, "", 0) }
506 |   );
507 | }
508 |
509 | if (path === "/api/me" && method === "GET") {
510 |   return json({ user: publicUser(user) });
511 | }
512 |
513 | if (path === "/api/classes" && method === "GET") {
514 |   const result = await env.DB.prepare(`
515 |     SELECT c.id, c.name, c.teacher_id, cm.muted, cm.last_read_seq
516 |     FROM classes c
517 |     JOIN class_members cm ON cm.class_id = c.id
518 |     WHERE cm.user_id = ?
519 |     ORDER BY c.created_at
520 |   `).bind(user.id).all();
521 |
522 |   return json({ classes: result.results });
523 | }
524 |
525 | if (path === "/api/classes" && method === "POST") {
526 |   if (user.role !== "teacher") fail(403, "Only teachers can create classes.");
527 |
528 |   await consume(env, `class-create:${user.id}`, 10, 86400);
529 | }

```

```

530 |     const body = await bodyJSON(request);
531 |     const name = cleanText(body.name, 1, 80, "Class name");
532 |     const classId = crypto.randomUUID();
533 |     const code = randomToken(10).toUpperCase();
534 |
535 |     await env.DB.batch([
536 |       env.DB.prepare(`
537 |         INSERT INTO classes(id, name, teacher_id, join_hash, created_at)
538 |         VALUES (?, ?, ?, ?, ?)
539 |       `).bind(classId, name, user.id, await hash(code), Date.now()),
540 |       env.DB.prepare(`
541 |         INSERT INTO class_members(class_id, user_id, joined_at)
542 |         VALUES (?, ?, ?)
543 |       `).bind(classId, user.id, Date.now())
544 |     ]);
545 |
546 |     return json({ id: classId, name, code }, 201);
547 |   }
548 |
549 |   if (path === "/api/classes/join" && method === "POST") {
550 |     await consume(env, `join-existing:${user.id}`, 10, 3600);
551 |
552 |     const body = await bodyJSON(request);
553 |     const codeHash = await hash(normalizeInvite(body.code));
554 |
555 |     const classroom = await env.DB.prepare(
556 |       "SELECT id FROM classes WHERE join_hash = ?"
557 |     ).bind(codeHash).first();
558 |
559 |     if (!classroom) fail(400, "Invalid invitation code.");
560 |
561 |     const result = await env.DB.prepare(`
562 |       INSERT OR IGNORE INTO class_members(class_id, user_id, joined_at)
563 |       SELECT c.id, ?, ? FROM classes c
564 |       WHERE c.id = ? AND c.join_hash = ?
565 |       AND NOT EXISTS (
566 |         SELECT 1 FROM removed_members WHERE class_id = c.id AND user_id = ?
567 |       )
568 |       AND (SELECT COUNT(*) FROM class_members WHERE class_id = c.id) < 41
569 |     `).bind(user.id, Date.now(), classroom.id, codeHash, user.id).run();
570 |
571 |     if (!result.meta.changes && !(await membership(env, classroom.id, user.id))) {
572 |       fail(403, "You cannot join this class.");
573 |     }
574 |
575 |     return json({ id: classroom.id });
576 |   }
577 |
578 |   const ws = path.match(/^\/ws\/classes\/([a-f0-9-]+)$/);
579 |   if (ws && method === "GET") {
580 |     await consume(env, `connect:${user.id}`, 60, 3600);
581 |     return roomRequest(request, env, ws[1]);
582 |   }
583 |
584 |   const room = path.match(
585 |     /^\/api\/classes\/([a-f0-9-]+)\/(roster|invite|members|messages)(?:\/([a-f0-9-]+))?$/
586 |   );
587 |
588 |   if (room) {

```

```

589 |     await consume(env, `room-api:${user.id}`, 240, 3600);
590 |     return roomRequest(request, env, room[1]);
591 |   }
592 |
593 |   if (path === "/api/uploads" && method === "POST") {
594 |     return upload(request, env, user);
595 |   }
596 |
597 |   const file = path.match(/^\/api\/files\/([a-f0-9-]+)$/);
598 |   if (file && method === "GET") {
599 |     return download(request, env, user, file[1]);
600 |   }
601 |
602 |   fail(404, "Not found.");
603 | }
604 |
605 | export default {
606 |   async fetch(request, env) {
607 |     try {
608 |       return await handle(request, env);
609 |     } catch (error) {
610 |       return errorResponse(error);
611 |     }
612 |   },
613 |
614 |   async scheduled(_event, env, ctx) {
615 |     ctx.waitUntil(cleanup(env));
616 |   }
617 | };
618 |
619 | async function cleanup(env) {
620 |   const day = 86400000;
621 |   const now = Date.now();
622 |
623 |   // At most 100 object removals per run.
624 |   const stale = await env.DB.prepare(`
625 |     SELECT a.id, a.r2_key
626 |     FROM attachments a
627 |     WHERE a.created_at < ?
628 |     OR (
629 |       a.created_at < ?
630 |       AND NOT EXISTS (
631 |         SELECT 1 FROM messages m
632 |         WHERE m.attachment_id = a.id AND m.deleted = 0
633 |       )
634 |     )
635 |     ORDER BY a.created_at
636 |     LIMIT 100
637 | `).bind(now - 30 * day, now - day).all();
638 |
639 |   for (const item of stale.results) {
640 |     await env.FILES.delete(item.r2_key);
641 |     await env.DB.prepare("DELETE FROM attachments WHERE id = ?")
642 |       .bind(item.id).run();
643 |   }
644 |
645 |   await env.DB.batch([
646 |     env.DB.prepare("DELETE FROM sessions WHERE expires_at <= ?").bind(now),
647 |     env.DB.prepare("DELETE FROM messages WHERE created_at < ?")

```

```

648 |     .bind(now - 30 * day)
649 |   });
650 | }
651 |
652 | /**
653 |  * Private, SQLite-backed quota coordinator.
654 |  * Persistent counters survive Durable Object hibernation/restarts.
655 |  */
656 | export class BudgetGuard {
657 |   constructor(ctx) {
658 |     this.ctx = ctx;
659 |
660 |     ctx.storage.sql.exec(`
661 |       CREATE TABLE IF NOT EXISTS counters (
662 |         key TEXT PRIMARY KEY,
663 |         used INTEGER NOT NULL,
664 |         expires INTEGER NOT NULL
665 |       );
666 |       CREATE INDEX IF NOT EXISTS counters_expiry ON counters(expires);
667 |     `);
668 |   }
669 |
670 |   async fetch(request) {
671 |     const { key, limit, seconds, amount } = await request.json();
672 |     const sql = this.ctx.storage.sql;
673 |     const now = Date.now();
674 |
675 |     sql.exec("DELETE FROM counters WHERE expires > 0 AND expires <= ?", now);
676 |
677 |     const row = [...sql.exec(
678 |       "SELECT used FROM counters WHERE key = ?", key
679 |     )][0];
680 |
681 |     if ((row?.used || 0) + amount > limit) {
682 |       return new Response("Quota exceeded", { status: 429 });
683 |     }
684 |
685 |     sql.exec(`
686 |       INSERT INTO counters(key, used, expires) VALUES (?, ?, ?)
687 |       ON CONFLICT(key) DO UPDATE SET used = used + excluded.used
688 |     `, key, amount, seconds ? now + seconds * 1000 : 0);
689 |
690 |     return new Response("OK");
691 |   }
692 | }
693 |
694 | /**
695 |  * One SQLite-backed, hibernating Durable Object per class.
696 |  */
697 | export class Classroom {
698 |   constructor(ctx, env) {
699 |     this.ctx = ctx;
700 |     this.env = env;
701 |     this.queue = Promise.resolve();
702 |
703 |     ctx.storage.sql.exec(`
704 |       CREATE TABLE IF NOT EXISTS rate_limits (
705 |         user_id TEXT PRIMARY KEY,
706 |         started INTEGER NOT NULL,

```

```

707 |         count INTEGER NOT NULL
708 |     );
709 | `);
710 | }
711 |
712 | serial(work) {
713 |     const result = this.queue.then(work);
714 |     this.queue = result.catch(() => {});
715 |     return result;
716 | }
717 |
718 | send(socket, value) {
719 |     try {
720 |         socket.send(JSON.stringify(value));
721 |     } catch {
722 |         try { socket.close(1011, "Connection error"); } catch {}
723 |     }
724 | }
725 |
726 | broadcast(value) {
727 |     for (const socket of this.ctx.getWebSockets()) {
728 |         this.send(socket, value);
729 |     }
730 | }
731 |
732 | presence() {
733 |     const users = new Set();
734 |
735 |     for (const socket of this.ctx.getWebSockets()) {
736 |         const auth = socket.deserializeAttachment();
737 |         if (auth?.expires_at > Date.now()) users.add(auth.id);
738 |     }
739 |
740 |     this.broadcast({ type: "presence", userIds: [...users] });
741 | }
742 |
743 | rateLimit(userId) {
744 |     const sql = this.ctx.storage.sql;
745 |     const now = Date.now();
746 |     const row = [...sql.exec(
747 |         "SELECT started, count FROM rate_limits WHERE user_id = ?", userId
748 |     )][0];
749 |
750 |     if (!row || now - row.started >= 60000) {
751 |         sql.exec(`
752 |             INSERT INTO rate_limits(user_id, started, count) VALUES (?, ?, 1)
753 |             ON CONFLICT(user_id) DO UPDATE SET started = excluded.started, count = 1
754 |             `, userId, now);
755 |         return;
756 |     }
757 |
758 |     if (row.count >= 90) fail(429, "Too many events. Wait a minute.");
759 |     sql.exec("UPDATE rate_limits SET count = count + 1 WHERE user_id = ?", userId);
760 | }
761 |
762 | async fetch(request) {
763 |     try {
764 |         return await this.serial(() => this.route(request));
765 |     } catch (error) {

```

```

766 |     return ErrorResponse(error);
767 | }
768 | }
769 |
770 | async route(request) {
771 |     const url = new URL(request.url);
772 |     const user = await authenticate(request, this.env);
773 |     const match = url.pathname.match(/\s/classes\/([a-f0-9-]+)/);
774 |     const classId = match?.[1];
775 |
776 |     const member = await membership(this.env, classId, user.id);
777 |     if (!member) fail(403, "You are not a member of this class.");
778 |
779 |     if (url.pathname.startsWith("/ws/")) {
780 |         if (request.headers.get("Upgrade)?.toLowerCase() !== "websocket") {
781 |             fail(426, "WebSocket required.");
782 |         }
783 |
784 |         const sockets = this.ctx.getWebSockets();
785 |         if (sockets.length >= 82) fail(429, "Class connection limit reached.");
786 |
787 |         const ownSockets = sockets.filter(
788 |             socket => socket.deserializeAttachment()?.id === user.id
789 |         );
790 |
791 |         if (ownSockets.length >= 2) {
792 |             ownSockets[0].close(4001, "Opened on another tab or device.");
793 |         }
794 |
795 |         const pair = new WebSocketPair();
796 |         const client = pair[0];
797 |         const server = pair[1];
798 |
799 |         this.ctx.acceptWebSocket(server);
800 |         server.serializeAttachment({
801 |             id: user.id,
802 |             display_name: user.display_name,
803 |             session_hash: user.session_hash,
804 |             expires_at: user.expires_at,
805 |             classId
806 |         });
807 |
808 |         this.send(server, { type: "ready" });
809 |         this.presence();
810 |
811 |         return new Response(null, { status: 101, websocket: client });
812 |     }
813 |
814 |     const suffix = url.pathname.split(`/classes/${classId}/`)[1];
815 |     const [resource, targetId] = suffix.split("/");
816 |     const method = request.method;
817 |
818 |     if (resource === "roster" && method === "GET") {
819 |         const result = await this.env.DB.prepare(`
820 |             SELECT u.id, u.display_name, u.role, cm.muted, cm.last_read_seq
821 |             FROM class_members cm
822 |             JOIN users u ON u.id = cm.user_id
823 |             WHERE cm.class_id = ?
824 |             ORDER BY u.role DESC, u.display_name

```

```

825 |     `).bind(classId).all();
826 |
827 |     return json({ members: result.results });
828 | }
829 |
830 | if (resource === "messages" && method === "GET") {
831 |     const afterValue = url.searchParams.get("after");
832 |     const beforeValue = url.searchParams.get("before");
833 |     let query;
834 |     let args;
835 |
836 |     if (afterValue !== null) {
837 |         const after = Number(afterValue);
838 |         if (!Number.isSafeInteger(after) || after < 0) fail(400, "Invalid cursor.");
839 |
840 |         query = `${MESSAGE_SELECT}
841 |             WHERE m.class_id = ? AND m.seq > ?
842 |             ORDER BY m.seq ASC LIMIT 100`;
843 |         args = [classId, after];
844 |     } else if (beforeValue !== null) {
845 |         const before = Number(beforeValue);
846 |         if (!Number.isSafeInteger(before) || before < 1) fail(400, "Invalid cursor.");
847 |
848 |         query = `${MESSAGE_SELECT}
849 |             WHERE m.class_id = ? AND m.seq < ?
850 |             ORDER BY m.seq DESC LIMIT 100`;
851 |         args = [classId, before];
852 |     } else {
853 |         query = `${MESSAGE_SELECT}
854 |             WHERE m.class_id = ? ORDER BY m.seq DESC LIMIT 100`;
855 |         args = [classId];
856 |     }
857 |
858 |     const result = await this.env.DB.prepare(query).bind(...args).all();
859 |     const messages = result.results.map(messageJSON);
860 |
861 |     if (afterValue === null) messages.reverse();
862 |
863 |     return json({ messages });
864 | }
865 |
866 | if (member.teacher_id !== user.id) {
867 |     fail(403, "Only the class teacher can perform this action.");
868 | }
869 |
870 | if (resource === "invite" && method === "POST") {
871 |     const code = randomToken(10).toUpperCase();
872 |
873 |     await this.env.DB.prepare(
874 |         "UPDATE classes SET join_hash = ? WHERE id = ?"
875 |     ).bind(await hash(code), classId).run();
876 |
877 |     return json({ code });
878 | }
879 |
880 | if (resource === "members" && targetId && method === "PATCH") {
881 |     if (targetId === user.id) fail(400, "You cannot mute yourself.");
882 |
883 |     const body = await bodyJSON(request);

```

```

884 |     if (typeof body.muted !== "boolean") fail(400, "Invalid mute value.");
885 |
886 |     await this.env.DB.prepare(`
887 |         UPDATE class_members SET muted = ?
888 |         WHERE class_id = ? AND user_id = ?
889 |     `).bind(body.muted ? 1 : 0, classId, targetId).run();
890 |
891 |     this.broadcast({ type: "roster:changed" });
892 |     return json({ ok: true });
893 | }
894 |
895 | if (resource === "members" && targetId && method === "DELETE") {
896 |     if (targetId === user.id) fail(400, "You cannot remove yourself.");
897 |
898 |     // Removing someone rotates the shared invitation as well.
899 |     const code = randomToken(10).toUpperCase();
900 |
901 |     await this.env.DB.batch([
902 |         this.env.DB.prepare(`
903 |             INSERT OR REPLACE INTO removed_members(class_id, user_id, removed_at)
904 |             SELECT class_id, user_id, ? FROM class_members
905 |             WHERE class_id = ? AND user_id = ?
906 |         `).bind(Date.now(), classId, targetId),
907 |         this.env.DB.prepare(`
908 |             DELETE FROM class_members WHERE class_id = ? AND user_id = ?
909 |         `).bind(classId, targetId),
910 |         this.env.DB.prepare("UPDATE classes SET join_hash = ? WHERE id = ?")
911 |         .bind(await hash(code), classId)
912 |     ]);
913 |
914 |     for (const socket of this.ctx.getWebSockets()) {
915 |         if (socket.deserializeAttachment()?.id === targetId) {
916 |             socket.close(4003, "Removed from class.");
917 |         }
918 |     }
919 |
920 |     this.broadcast({ type: "roster:changed" });
921 |     this.presence();
922 |
923 |     return json({ ok: true, code });
924 | }
925 |
926 | if (resource === "messages" && targetId && method === "DELETE") {
927 |     await this.env.DB.prepare(`
928 |         UPDATE messages SET deleted = 1, content = ''
929 |         WHERE id = ? AND class_id = ?
930 |     `).bind(targetId, classId).run();
931 |
932 |     this.broadcast({ type: "message:deleted", id: targetId });
933 |     return json({ ok: true });
934 | }
935 |
936 | fail(404, "Not found.");
937 | }
938 |
939 | async websocketMessage(socket, raw) {
940 |     return this.serial(async () => {
941 |         let event;
942 |

```

```

943 |     try {
944 |       if (typeof raw !== "string" || raw.length > 8192) {
945 |         fail(400, "Invalid message frame.");
946 |       }
947 |
948 |       event = JSON.parse(raw);
949 |       if (!event || typeof event !== "object") fail(400, "Invalid event.");
950 |
951 |       const auth = socket.deserializeAttachment();
952 |       if (!auth || auth.expires_at <= Date.now()) {
953 |         socket.close(4001, "Session expired.");
954 |         return;
955 |       }
956 |
957 |       this.rateLimit(auth.id);
958 |
959 |       if (event.type === "typing") {
960 |         this.broadcast({
961 |           type: "typing",
962 |           userId: auth.id,
963 |           name: auth.display_name
964 |         });
965 |         return;
966 |       }
967 |
968 |       // Persistent operations always revalidate both session and membership.
969 |       const validSession = await this.env.DB.prepare(`
970 |         SELECT 1 FROM sessions
971 |         WHERE token_hash = ? AND user_id = ? AND expires_at > ?
972 |       `).bind(auth.session_hash, auth.id, Date.now()).first();
973 |
974 |       if (!validSession) {
975 |         socket.close(4001, "Session expired.");
976 |         return;
977 |       }
978 |
979 |       const member = await membership(this.env, auth.classId, auth.id);
980 |       if (!member) {
981 |         socket.close(4003, "Class access revoked.");
982 |         return;
983 |       }
984 |
985 |       if (event.type === "message:new") {
986 |         if (member.muted) fail(403, "You are muted in this class.");
987 |
988 |         const clientId = cleanText(event.clientId, 1, 100, "Client ID");
989 |         const content = cleanText(event.content ?? "", 0, 4000, "Message");
990 |         const attachmentId = event.attachmentId || null;
991 |
992 |         const existing = await this.env.DB.prepare(`
993 |           SELECT id FROM messages
994 |           WHERE class_id = ? AND sender_id = ? AND client_id = ?
995 |         `).bind(auth.classId, auth.id, clientId).first();
996 |
997 |         if (existing) {
998 |           this.send(socket, {
999 |             type: "message:ack",
1000 |             clientId,
1001 |             message: await loadMessage(this.env, existing.id)

```

```

1002 |     });
1003 |     return;
1004 | }
1005 |
1006 | if (!content && !attachmentId) fail(400, "Write a message or attach a file.");
1007 |
1008 | if (attachmentId) {
1009 |     const attachment = await this.env.DB.prepare(`
1010 |         SELECT id FROM attachments
1011 |         WHERE id = ? AND class_id = ? AND owner_id = ?
1012 |         AND NOT EXISTS (
1013 |             SELECT 1 FROM messages WHERE attachment_id = attachments.id
1014 |         )
1015 |     `).bind(attachmentId, auth.classId, auth.id).first();
1016 |
1017 |     if (!attachment) fail(400, "Attachment is unavailable or already used.");
1018 | }
1019 |
1020 | const id = crypto.randomUUID();
1021 |
1022 | // Persistence happens BEFORE acknowledgement or broadcast.
1023 | await this.env.DB.prepare(`
1024 |     INSERT INTO messages
1025 |     (id, class_id, sender_id, client_id, content, attachment_id, created_at)
1026 |     VALUES (?, ?, ?, ?, ?, ?, ?)
1027 | `).bind(
1028 |     id, auth.classId, auth.id, clientId, content, attachmentId, Date.now()
1029 | ).run();
1030 |
1031 | const message = await loadMessage(this.env, id);
1032 |
1033 | this.send(socket, { type: "message:ack", clientId, message });
1034 | this.broadcast({ type: "message:new", message });
1035 | return;
1036 | }
1037 |
1038 | if (event.type === "message:read") {
1039 |     const seq = Number(event.seq);
1040 |     if (!Number.isSafeInteger(seq) || seq < 1) fail(400, "Invalid read cursor.");
1041 |
1042 |     const exists = await this.env.DB.prepare(`
1043 |         SELECT 1 FROM messages WHERE class_id = ? AND seq = ?
1044 |     `).bind(auth.classId, seq).first();
1045 |
1046 |     if (!exists) fail(400, "Unknown message.");
1047 |
1048 |     await this.env.DB.prepare(`
1049 |         UPDATE class_members
1050 |         SET last_read_seq = MAX(last_read_seq, ?)
1051 |         WHERE class_id = ? AND user_id = ?
1052 |     `).bind(seq, auth.classId, auth.id).run();
1053 |
1054 |     this.broadcast({
1055 |         type: "message:read",
1056 |         userId: auth.id,
1057 |         seq: Math.max(seq, member.last_read_seq)
1058 |     });
1059 |     return;
1060 | }

```

```

1061 |
1062 |     fail(400, "Unsupported event.");
1063 | } catch (error) {
1064 |     if (!(error instanceof HttpError)) console.error(error);
1065 |
1066 |     this.send(socket, {
1067 |         type: "error",
1068 |         clientId: event?.clientId,
1069 |         code: error instanceof HttpError ? error.status : 500,
1070 |         message:
1071 |             error instanceof HttpError ? error.message : "Could not save message."
1072 |     });
1073 | }
1074 | });
1075 | }
1076 |
1077 | async websocketClose(socket, code, reason) {
1078 |     try { socket.close(code, reason); } catch {}
1079 |     this.presence();
1080 | }
1081 |
1082 | async websocketError(socket) {
1083 |     try { socket.close(1011, "Connection error"); } catch {}
1084 |     this.presence();
1085 | }
1086 | }

```

=== backend/wrangler.toml === (35 lines, 723 bytes)

```
1 | name = "classconnect-api"
2 | main = "src/index.js"
3 | compatibility_date = "2025-04-01"
4 |
5 | # Reach the API through the Pages service binding, not a public workers.dev URL.
6 | workers_dev = false
7 | preview_urls = false
8 |
9 | [vars]
10 | APP_ORIGIN = "https://classconnect.pages.dev"
11 |
12 | [[d1_databases]]
13 | binding = "DB"
14 | database_name = "classconnect-db"
15 | database_id = "REPLACE_WITH_D1_DATABASE_ID"
16 | migrations_dir = "migrations"
17 |
18 | [[r2_buckets]]
19 | binding = "FILES"
20 | bucket_name = "classconnect-files"
21 |
22 | [[durable_objects.bindings]]
23 | name = "ROOMS"
24 | class_name = "ClassRoom"
25 |
26 | [[durable_objects.bindings]]
27 | name = "GUARD"
28 | class_name = "BudgetGuard"
29 |
30 | [[migrations]]
31 | tag = "v1"
32 | new_sqlite_classes = ["ClassRoom", "BudgetGuard"]
33 |
34 | [triggers]
35 | crons = ["17 3 * * *"]
```

=== backend/migrations/0001_initial.sql === (73 lines, 2305 bytes)

```
1 | PRAGMA foreign_keys = ON;
2 |
3 | CREATE TABLE users (
4 |   id TEXT PRIMARY KEY,
5 |   display_name TEXT NOT NULL,
6 |   role TEXT NOT NULL CHECK (role IN ('teacher', 'student')),
7 |   recovery_hash TEXT NOT NULL UNIQUE,
8 |   created_at INTEGER NOT NULL
9 | );
10 |
11 | CREATE TABLE sessions (
12 |   token_hash TEXT PRIMARY KEY,
13 |   user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
14 |   expires_at INTEGER NOT NULL
15 | );
16 |
17 | CREATE INDEX sessions_expiry ON sessions(expires_at);
18 | CREATE INDEX sessions_user ON sessions(user_id);
19 |
20 | CREATE TABLE classes (
21 |   id TEXT PRIMARY KEY,
22 |   name TEXT NOT NULL,
23 |   teacher_id TEXT NOT NULL REFERENCES users(id),
24 |   join_hash TEXT NOT NULL UNIQUE,
25 |   created_at INTEGER NOT NULL
26 | );
27 |
28 | CREATE TABLE class_members (
29 |   class_id TEXT NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
30 |   user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
31 |   joined_at INTEGER NOT NULL,
32 |   muted INTEGER NOT NULL DEFAULT 0 CHECK (muted IN (0, 1)),
33 |   last_read_seq INTEGER NOT NULL DEFAULT 0,
34 |   PRIMARY KEY (class_id, user_id)
35 | );
36 |
37 | CREATE INDEX memberships_user ON class_members(user_id, class_id);
38 |
39 | CREATE TABLE attachments (
40 |   id TEXT PRIMARY KEY,
41 |   class_id TEXT NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
42 |   owner_id TEXT NOT NULL REFERENCES users(id),
43 |   r2_key TEXT NOT NULL UNIQUE,
44 |   file_name TEXT NOT NULL,
45 |   mime_type TEXT NOT NULL,
46 |   size_bytes INTEGER NOT NULL CHECK (size_bytes > 0),
47 |   created_at INTEGER NOT NULL
48 | );
49 |
50 | CREATE INDEX attachments_expiry ON attachments(created_at);
51 |
52 | CREATE TABLE messages (
53 |   seq INTEGER PRIMARY KEY AUTOINCREMENT,
54 |   id TEXT NOT NULL UNIQUE,
55 |   class_id TEXT NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
56 |   sender_id TEXT NOT NULL REFERENCES users(id),
57 |   client_id TEXT NOT NULL,
```

```
58 | content TEXT NOT NULL DEFAULT '',
59 | attachment_id TEXT UNIQUE REFERENCES attachments(id) ON DELETE SET NULL,
60 | created_at INTEGER NOT NULL,
61 | deleted INTEGER NOT NULL DEFAULT 0 CHECK (deleted IN (0, 1)),
62 | UNIQUE (class_id, sender_id, client_id)
63 | );
64 |
65 | CREATE INDEX messages_class_seq ON messages(class_id, seq);
66 | CREATE INDEX messages_expiry ON messages(created_at);
67 |
68 | CREATE TABLE removed_members (
69 | class_id TEXT NOT NULL REFERENCES classes(id) ON DELETE CASCADE,
70 | user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
71 | removed_at INTEGER NOT NULL,
72 | PRIMARY KEY (class_id, user_id)
73 | );
```